

Onre - Re-Audit - 2

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	RWA	Documentation quality	Medium
Timeline	2025-12-09 through 2025-12-20	Test quality	High
Language	Rust	Total Findings	15 Fixed: 10 Acknowledged: 5
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	1 Fixed: 1
Specification	None	Medium severity findings ⓘ	3 Fixed: 2 Acknowledged: 1
Source Code	onre-finance/onre-sol ⓘ #b909c7f ⓘ	Low severity findings ⓘ	5 Fixed: 2 Acknowledged: 3
Auditors	- Hamed Mohammadi Auditing Engineer - Darren Jensen Auditing Engineer - Paul Clemson Auditing Engineer	Undetermined severity ⓘ	0

ty findin gs	
Infor matio nal findin gs	6  Fixed: 5 Acknowledged: 1

Summary of Findings

The **Onre** protocol is a Solana program built with the Anchor framework that facilitates the sale of real world reinsurance assets through an offers-based system, enabling depositors to earn institutional-grade yield backed by real world reinsurance. The protocol allows the **boss** to create offers by defining the buy and sell tokens, along with price vectors, while users can choose to fill these offers either fully or partially.

The current iteration introduces **OfferVector** to improve price management. Through **OfferVector**, the **boss** can influence how token prices evolve over time with respect to the APR, enabling a continuous price curve.

The protocol also includes a redemption mechanism. This allows the **boss** to create a **RedemptionOffer** that specifies the buy and sell tokens and references the original **Offer** it is based on. Other users can then create a signed **RedemptionRequest**, which are processed off-chain and later fulfilled on-chain by an authorized user once sufficient liquidity becomes available, subject to protocol owner considerations.

Overall, the protocol is well written, and the code quality reflects a strong commitment to security and safety. The codebase is generally well structured and clearly documented, with refactoring across iterations leading to meaningful improvements in readability, modularity, and separation of concerns. Core flows such as offer execution, redemptions, pricing, and accounting are implemented in a consistent and auditable manner, supported by clear access control checks, shared utilities, and helpful inline comments and events that improve transparency and off-chain observability.

During the audit we identified a small number of issues, including one issue, **ONR-1**, with a potentially high impact on protocol operations and therefore classified as High severity.

Fix-Review Update 2026-01-07:

All issues identified during the audit have been fixed, mitigated, or acknowledged, and the provided fixes were reviewed. Additionally, during the fix review, a small change to the code was detected: the per-user **nonce** in the **create_redemption_request()** function was effectively replaced with a **request_id** and a **request_counter** that appear to achieve the same goal previously achieved by the **nonce**. These two new fields work across all requests and prevent the same signature from being used twice. This change, although checked by the auditors, falls outside the scope of the normal fix review.

ID	DESCRIPTION	SEVERITY	STATUS
ONR-1	Missing Association Checks Between Redemption Offer and Redemption Request Enables State Corruption	High ⓘ	Fixed
ONR-2	Denial of Service Risk Due to Inaccessible Mint Authority	Medium ⓘ	Acknowledged
ONR-3	The RedemptionOffer.requested_redemptions Field Not Updated upon a Successful Redemption Request Fulfillment	Medium ⓘ	Fixed
ONR-4	Consider Enforcing Pda Constraint Validation	Medium ⓘ	Fixed
ONR-5	Missing Slippage Protection Can Result in Less-than-Ideal Trade Execution	Low ⓘ	Acknowledged
ONR-6	RedemptionOffer.fee_basis_points Can Be Increased While a User's Request Is Active, Causing Them to Receive Less Tokens Out than Expected	Low ⓘ	Acknowledged
ONR-7	Ensure Distinct Accounts for Approver Roles	Low ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
ONR-8	Redemption Requests Can Still Be Made for RedemptionOffer Instances Where the Underlying Offer Has Been Closed	• Low ⓘ	Fixed
ONR-9	Permissionless Initialization Lets First Caller Seize Boss Role on Immutable Deployments	• Low ⓘ	Acknowledged
ONR-10	Kill Switch Not Enforced in Minting Logic Allows Token Inflation During Emergency Pause	• Informational ⓘ	Acknowledged
ONR-11	Perform a Statelessness Check for the Ed25519 Program	• Informational ⓘ	Fixed
ONR-12	RedemptionRequest Accounts Should Be Closed Once They Are No Longer Needed	• Informational ⓘ	Fixed
ONR-13	Floor Division During Fee Calculations	• Informational ⓘ	Fixed
ONR-14	Approval Message Not Strictly Bound to Signed Payload	• Informational ⓘ	Fixed
ONR-15	Redundant expires_at Parameter in CreateRedemptio	• Informational ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
	nRequest Due to Solana Replay Protection		

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:

1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Operational Considerations

- **Protocol Shutdown:** The `close_state` instruction allows the boss to permanently close the program's main state account and reclaim its rent balance. This operation effectively disables the entire protocol, as most instructions depend on the existence of the `state` account.
- **Unlimited Minting:** In `transfer_mint_authority_to_boss`, post-transfer the boss can actually perform unlimited minting (i.e., bypasses program max-supply logic). After the `transfer_mint_authority_to_boss` instruction, the boss becomes the direct mint authority of the token and can mint arbitrarily using raw SPL calls, ignoring `state.max_supply` and all in-program checks. This is clearly intended as an "emergency recovery" escape hatch, but it means the safety of supply caps and protocol guarantees fully depends on boss governance or policy once this is used.
- **Vault Withdrawal Risk:** `redemption_vault_withdraw` allows the boss to withdraw any mint held in the redemption vault ATA, which can deplete reserves intended for redemptions and create insolvency risk for pending requests.
- **Adjustable Supply Cap:** Even when `mint_to` enforces `max_supply`, the effective "cap" can be adjusted by the boss at any time by calling the `configure_max_supply` instruction.
- **Broad Approvals:** The current `ApprovalMessage ( {program_id, user_pubkey, expiry} )` supports broad approvals; operationally this means an approver signature can authorize arbitrary offers or amounts until expiry.
- **Reporting Accuracy:** `get_circulating_supply` and `get_tvl` treat an uninitialized vault ATA as a zero balance and subtract vault balances from mint supply; operationally, incorrect vault selection or token program mismatches can skew reporting and dashboards.
- **Redemption Liquidity:** The protocol team is responsible for ensuring the necessary assets to fulfill redemption requests are made available as soon as possible to ensure users can redeem in a timely manner.
- **Redemption Manipulation:** Users with particularly large stakes could initiate a large redemption request, triggering the protocol team to begin liquidating assets, and then canceling their redemption request.
- **Redemption Control:** `fulfill_redemption_request` requires only a `redemption_admin` signature; users cannot force completion. If the backend is down, misconfigured, or compromised, redemptions can be delayed, selectively processed, or processed incorrectly.

- **Redemption Execution:** Fulfilment of the redemption requests are subject to the protocol team actually performing the redemptions and not taking users' liquidity hostage.

Key Actors And Their Capabilities

Boss (`State.boss`)

- The primary super-admin of the protocol.
- Represents the highest-trust actor in the system.
- Has full control over program configuration and lifecycle, including:
 - Setting and clearing admins.
 - Updating critical roles (e.g., redemption admin).
 - Transferring mint authority between the program and itself.
 - Minting tokens when the program holds mint authority.
 - Withdrawing tokens from vaults.
 - Activating emergency kill switches.
 - Permanently closing the program state.

Future Boss (`State.proposed_boss`)

- The account designated to assume the boss role after accepting it.
- Assigned by the current boss and must accept the role before performing any other operations.

Admins (`State.admins[]`)

- A limited set of accounts stored in program state.
- Authorized to perform restricted operational actions (e.g., enabling the kill switch).
- Cannot override or replace the boss.
- Intended as delegated operational or emergency responders.

Redemption Admin (`State.redemption_admin`)

- A specialized privileged role responsible for processing and fulfilling redemption requests.
- Signs and executes backend-driven redemption flows.
- Does not require redeemer participation at fulfillment time.
- Trusted to act honestly and promptly on behalf of users.

Approvers (`State.approver1` , `State.approver2`)

- Two trusted off-chain signing authorities.
- Provide cryptographic approvals (via Ed25519 signatures) for offers that require explicit authorization.
- Approvals are time-bound and program-scoped.
- Used to gate permissionless offer execution when required by the offer configuration.

Backend Service

- An off-chain service operated by the protocol.

- Plays a critical role in liveness and execution timing.
- Implicitly trusted to follow protocol rules and user intent.
- Submits and signs transactions for redemption fulfillment, offer execution, and maintenance tasks.

Program-Derived Authorities (PDAs)

- On-chain program-controlled accounts used for secure custody and authority including:
 - Mint authority PDA.
 - Offer vault authority PDA.
 - Redemption vault authority PDA.
 - Permissionless intermediary authority PDA.
- Ensures that sensitive token operations are only executable by the program logic.

Users / Traders

- End users interacting with offers and redemptions.
- Provide input tokens and receive output tokens.
- May interact via permissionless or approval-gated flows.
- Do not hold administrative or minting authority.

Findings

ONR-1

Missing Association Checks Between Redemption Offer and Redemption Request Enables State Corruption

• High ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `9b917d213718092c0d416835a00f1fedef079c8a`.

The client provided the following explanation:

Added missing association checks.

File(s) affected:

`programs/onreapp/src/instructions/redemption/cancel_redemption_request.rs`

Description: The `cancel_redemption_request` instruction accepts both `redemption_offer` and `redemption_request` as mutable accounts but does not enforce any relationship between them. There is no validation that the provided redemption request belongs to the specified redemption offer.

Exploit Scenario:

An authorized signer (redeemer, redemption_admin, or boss) submits a transaction with a valid pending `redemption_request` tied to Offer A, but supplies a different `redemption_offer` (Offer B). The instruction successfully cancels the request and subtracts the request amount from Offer B's `requested_redemptions`, corrupting its accounting state. This can lead to incorrect redemption limits, blocked redemptions, or underflow-related denial of service.

Recommendation: Enforce strict association between redemption offers and requests by adding explicit constraints that should be enforced at the account validation level to guarantee state consistency and prevent cross-offer manipulation.

ONR-2

Denial of Service Risk Due to Inaccessible Mint Authority

• Medium ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

The `transfer_mint_authority_to_*` instructions are helper utilities intended to simplify mint authority transfers and reduce the risk of errors that can occur when performing this operation manually (for example, via the Solana SPL Token CLI). These helpers are provided for convenience only. Mint authority can still be transferred manually if needed, ensuring that control over the token mint is not lost in scenarios such as the boss account being migrated or replaced.

File(s) affected:

```
programs/onreapp/src/instructions/mint_authority/transfer_mint_authority_to_program.rs
```

Description: Several instructions in the program require mint authority to mint or burn tokens as part of their normal operation. This mint authority can be either a program-derived address or the `boss` account. However, since the program allows transferring ownership of the `boss` role, a boss transfer performed without properly updating the mint authorities of the relevant token mints can leave the program unable to mint or burn tokens. This would effectively result in a denial of service.

Additionally, the `TransferMintAuthorityToProgram` instruction requires the current mint authority to be the active `boss` before allowing the authority to be transferred to a PDA. If the previous `boss` becomes inaccessible after boss transfer is performed, there is no mechanism to move the mint authority to the new `boss` or to the PDA, permanently blocking mint and burn operations.

Recommendation: To mitigate this risk, consider modifying the `AcceptBoss` instruction to require that the mint authority is first set to a PDA controlled by the program before finalizing the `boss` transfer. Once the transfer is complete, the new `boss` can optionally reclaim control using the `TransferMintAuthorityToBoss` instruction if direct ownership is required.

If setting the mint authority to a PDA during the boss transfer is not feasible, an alternative approach is to introduce a temporary admin account. In this flow, the current `boss` would first transfer mint authority to the temporary admin, then initiate the boss transfer. Before accepting the role, the new `boss` would verify that the temporary admin, not the previous boss, holds the mint authority. After the boss transfer is finalized, the temporary admin can transfer the mint authority to the new `boss`. This approach ensures continuity of mint control throughout the transfer process and prevents accidental denial of service.

ONR-3

The `RedemptionOffer.requested_redemptions` Field Not Updated upon a Successful Redemption Request Fulfillment

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `84fd65216f0a6133ada8eaa65625fdb9ddf0deee`.

The client provided the following explanation:

Added subtracting requested redemptions when fulfilling a redemption request

File(s) affected:

```
programs/onreapp/src/instructions/redemption/fulfill_redemption_request.rs
```

Description: The `RedemptionOffer` account holds the state related to a given redemption offer, including the current `requested_redemptions` amount. This value is added to when a user creates a redemption request; however, it is not subtracted from when their redemption request is fulfilled. The inline documentation above the `fulfill_redemption_request()` function suggests that this action should take place, but the necessary logic is missing in the current implementation. This causes inconsistent accounting, making it more difficult for the protocol team to correctly ensure they provide the necessary amount of tokens to fulfill all requests.

Recommendation: Ensure that the `requested_redemptions` field has `token_in_amount` deducted from it within the logic of the `fulfill_redemption_request()` function.

ONR-4

Consider Enforcing Pda Constraint Validation

• Medium ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `a57cc029014452eb31ab13a5478c4d75edd86991` .

The client provided the following explanation:

Added missing PDA seed constraints

File(s) affected: `programs/onreapp/src/instructions/redemption/*`

Description: The `MakeRedemptionOffer` instruction defines `redemption_offer` as a PDA. However, subsequent instructions that accept the `redemption_offer` account do not revalidate its PDA derivation, which may allow incorrect redemption offer accounts to be passed in.

Similarly, the `redemption_request` account is defined as a PDA in the `CreateRedemptionRequest` instruction, but its PDA constraints are not validated in other instructions that interact with it.

Recommendation: To improve safety and consistency, consider adding explicit PDA constraint validation for the `redemption_offer` and `redemption_request` accounts in the `CancelRedemptionRequest`, `CreateRedemptionRequest`, `FulfillRedemptionRequest`, and `UpdateRedemptionOfferFee` instructions. Additionally, perform a comprehensive review of the codebase to identify and remediate any other instances where PDA accounts are used without proper constraint verification. Adding the mentioned PDA constraint validations will also address ONR-1.

ONR-5

Missing Slippage Protection Can Result in Less-than-Ideal Trade Execution

• Low ⓘ

Acknowledged

ⓘ Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

This issue is acknowledged as an acceptable design trade-off given the protocol's pricing model. Prices are expected to be fixed for long intervals, typically one day, meaning the lack of explicit slippage protection only matters during the brief moment

when the price updates. In practice, this results in at most a one-second window per day where execution could differ from user expectations.

To further mitigate this at the UX level, the frontend may display a timer indicating when the next price change will occur, allowing users to avoid submitting transactions near a price transition. Given the low likelihood and limited impact, we consider this risk acceptable and do not enforce additional on-chain slippage checks

File(s) affected: `programs/onreapp/src/instructions/offer/take_offer.rs`,
`programs/onreapp/src/instructions/redemption/create_redemption_request.rs`

Description: The `take_offer` instruction allows users to execute an offer by specifying only the amount of tokens in. This instruction does not accept or enforce any slippage bounds such as a minimum acceptable `token_out` amount or maximum execution price.

Since the exchange rate is calculated at execution time using a chain state, users have no guarantee that the received amount of `token_out` matches their expectations at the time of transaction submission. This exposes users to adverse price movement without recourse.

Recommendation: Introduce explicit slippage protection by requiring users to specify a minimum acceptable `token_out` amount. The instruction should revert if the calculated output amount falls below this threshold.

ONR-6

`RedemptionOffer.fee_basis_points` Can Be Increased While a User's Request Is Active, Causing Them to Receive Less Tokens Out than Expected

• Low ⓘ

Acknowledged

ⓘ Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

This issue is acknowledged as it is consistent with the protocol's trust model. The protocol is intentionally built around a high level of trust in the boss role, which already has broad control over protocol parameters, including fees. Allowing `fee_basis_points` to change while redemption requests are pending is therefore an accepted governance decision rather than a flaw.

In practice, fees are expected to be stable and change very rarely, further reducing the likelihood of user impact. Given the explicit trust assumptions and operational expectations, we consider this behavior acceptable and do not enforce additional on-chain restrictions.

File(s) affected:

```
programs/onreapp/src/instructions/redemption/fulfill_redemption_request.rs
```

Description: In the current implementation of the deposit flow the redemption fee is calculated at the time the request is fulfilled. For larger redemption requests, which may be forced to wait before their request can be fulfilled, this may mean that the `fee_basis_points` taken out of their redemption has been changed by the admin between their initial request and its fulfillment, causing them to pay a larger amount in fees than they expected.

Recommendation: Consider storing the redemption offer's current `fee_basis_points` at the time the `RedemptionRequest` is created to avoid users paying unexpected fees.

ONR-7

Ensure Distinct Accounts for Approver Roles

• Low ⓘ

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `1ad7a02fcd0c913948727fe98c6e19a9b8b63785`.

The client provided the following explanation:

Added check to ensure that the two approvers must be distinct

File(s) affected:

```
programs/onreapp/src/instructions/state_operations/add_approver.rs ,  
programs/onreapp/src/instructions/state_operations/remove_approver.rs
```

Description: The `add_approver()` function does not verify whether the proposed approver address is already assigned to an existing approver slot. As a result, if any approver slot is empty, the function may assign an address that is already set as the other approver, leading to duplicate approver assignments.

Additionally, the `remove_approver()` function first checks whether `approver1` matches the address to be removed and returns immediately if it does, without checking `approver2`. If the same address was mistakenly assigned to both `approver1` and `approver2`, removing it would require calling the function twice.

Recommendation: Consider adding explicit validation in `add_approver()` to ensure that the new approver address is not already assigned to any existing approver slot.

Redemption Requests Can Still Be Made for Instances Where the Underlying Offer Has Been Closed

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: 1f00d721cfd20db873d25d4be5d4e863d8a784dc .

The client provided the following explanation:

We addressed this issue by removing the `close_offer` instruction entirely. As a result, Offer accounts can no longer be deleted, which guarantees that any `RedemptionOffer` referencing an Offer will always be able to access the underlying account and will not fail due to a missing Offer state.

We do not plan to remove offers in normal operation. In the rare case where an offer needs to be effectively disabled, this can be achieved by deleting all of its price vectors, rendering the offer unusable as pricing information will no longer be available. This also prevents related redemption offers from being fulfilled, which is an acceptable outcome under this scenario.

If explicit offer deactivation or cleanup is required in the future, it will be implemented with dedicated and well-defined logic.

File(s) affected:

```
programs/onreapp/src/instruccions/redemption/create_redemption_request.rs
```

Description: Functionality exists within the protocol to close an existing Offer account. In such a scenario, any `RedemptionOffer` that is tied to the closed offer will be unable to fulfill further redemption requests as the function will return an error when trying to read the now closed Offer state. However, the `create_redemption_request()` function will not cause an error if the offer stored in the `RedemptionOffer` instance does not exist, meaning redemption requests can be created that will not be fulfilled. The severity of this issue is downgraded because the `create_redemption_request()` action will require a signature from the `redemption_admin` who should handle validating that the underlying offer is still active. However, a more robust design would be helpful to remove this potential point of failure.

Recommendation: Consider adding a similar `close_redemption_offer()` function to close outstanding `RedemptionOffer` accounts. Then, when an offer is being closed, ensure that the `RedemptionOffer` instance is first closed before closing the Offer that it is linked to.

Permissionless Initialization Lets First Caller Seize Boss Role on Immutable Deployments

• Low ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

This issue is acknowledged based on our deployment assumptions. The program will not be deployed as immutable, ensuring that a valid upgrade authority is always present and enforced during initialization. As a result, the described scenario where an arbitrary first caller could seize the boss role does not apply to our intended deployment setup.

Additionally, even in the unlikely event of a misconfigured deployment, the impact would be limited to a newly deployed instance with no user funds or protocol state at risk. Given these constraints and the minimal practical impact, we consider this issue acceptable and acknowledge it rather than modifying the initialization logic.

File(s) affected:

```
programs/onreapp/src/instructions/initialization/initialize.rs
```

Description: The `initialize` instruction derives the program's upgrade authority via `get_upgrade_authority()` and only enforces that `boss` matches this authority when the result is `Some(pubkey)`. If the program is immutable (upgrade authority is `None` under the upgradeable loader) or deployed under the `bpf_loader`, the function performs no alternative authorization check and proceeds to set the boss to the Signer.

Since the `state` PDA is created deterministically with `init`, and if the `upgrade_authority` is `None`, then the first signer to call `initialize` will permanently set `state.boss` to themselves. Although an `ImmutableProgram` error exists, it is never used to prevent this path.

Exploit Scenario:

1. The program is deployed as immutable (or under `bpf_loader`) before `initialize` is called.
2. An attacker observes the deployment and front-runs the intended initializer by calling `initialize` first.
3. `get_upgrade_authority()` returns `Ok(None)`, so no authority check runs.
4. The attacker's signer is written as `state.boss`, granting them control over all boss gated operations indefinitely.

Recommendation: Require explicit authorization even when the upgrade authority is `None`. For example: - Reject initialization for immutable programs unless a predefined initializer key signs, **or** - Treat `None` as an `InitializeErrorCode::ImmutableProgram` error unless the transaction

includes a trusted authority, **or** - Store an expected initializer/boss pubkey as a constant and require it during `initialize`. Any approach should ensure initialization cannot be claimed by an arbitrary first caller.

ONR-10

Kill Switch Not Enforced in Minting Logic Allows Token Inflation During Emergency Pause

• Informational ⓘ

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

The `mint_to` instruction is restricted to the boss role and is provided as a convenience instruction with a low expected frequency of use. No user-facing flows rely on this instruction.

The purpose of the kill switch is to pause user trade and interaction activities, not to restrict administrative or emergency actions performed by the boss. Allowing the boss to mint while the kill switch is active is consistent with the protocol's trust model and operational assumptions.

File(s) affected: `programs/onreapp/src/instructions/mint_authority/mint_to.rs`

Description: The `mint_to` instruction allows the boss to mint new ONyc tokens but does not check the `state.is_killed` flag intended to act as a global kill switch. As a result, token minting remains possible even when the protocol is placed into a killed or emergency paused state.

Recommendation: Add an explicit check in the `state` account validation in the `MintTo` struct to ensure minting is disabled when the kill switch is engaged.

ONR-11

Perform a Statelessness Check for the Ed25519 Program

• Informational ⓘ

Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `243f227343fff1485f402dd1fede95a6466e76748`.

The client provided the following explanation:

Added additional check for empty accounts in the ED25519 instruction

File(s) affected: `programs/onreapp/src/utils/approver/approver_utils.rs`

Description: The `verify_approval_message_generic()` function verifies an Ed25519 signature by extracting the previous instruction via its index and checking that the program ID matches the `ed25519_program`. However, the Ed25519 program is stateless, meaning the instruction should not include any accounts.

Recommendation: To fully validate the signature, add an extra check to ensure the program is stateless:

```
require!(ix.accounts.is_empty(), ErrorCode::BadEd25519Accounts);
```

ONR-12

RedemptionRequest Accounts Should Be Closed Once They Are No Longer Needed

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `9b917d213718092c0d416835a00f1fedef079c8a`.

The client provided the following explanation:

Added closing of redemption request accounts when fulfilling or cancelling the requests.

File(s) affected:

`programs/onreapp/src/instructions/redemption/cancel_redemption_request.rs`,
`programs/onreapp/src/instructions/redemption/fulfill_redemption_request.rs`

Description: When requesting a redemption within the protocol, a `RedemptionRequest` account is created on behalf of the user to track all state tied to their outstanding request. Once this request is either cancelled (by the user or by an admin) or fulfilled by the redemption admin, the `RedemptionRequest` account is no longer used but does not get closed. This means that every redemption request has a non-refundable rent overhead that users must pay even though their storage requirements are only temporary.

Recommendation: Consider closing the `RedemptionRequest` account after the request has been executed or cancelled and refunding the stored rent fee to the `redeemer` account.

ONR-13

Floor Division During Fee Calculations

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `de3a3465661cb040219bae8bfff02a26513d4eb7`.

The client provided the following explanation:

Fixed to use ceiling division.

File(s) affected: `programs/onreapp/src/utils/token_utils.rs`

Description: The `calculate_fees()` function currently uses floor division when computing `token_fee_amount`. While the difference per transaction is small, repeated calls can cumulatively reduce the protocol's collected fees.

Recommendation: Consider using ceiling division instead to ensure the protocol does not lose any fee amounts.

ONR-14

Approval Message Not Strictly Bound to Signed Payload

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `69e132882e7afb492dd0b516432cd88f6eac6ab8`.

The client provided the following explanation:

Added equality requirement for approved message and message from parameters

File(s) affected: `programs/onreapp/src/utils/approver/approver_utils.rs`

Description: The approval verification logic reconstructs a signed approval message from the Ed25519 instruction (`signed_msg`) and separately accepts an `ApprovalMessage` argument (`msg`). While both messages are individually validated for `program_id`, `user_pubkey`, and

`expiry_unix`, the implementation does not enforce that the signed message content exactly matches the message passed into the instruction.

As a result, the cryptographic signature is not strictly bound to the instruction level message argument, relying instead on implicit assumptions about message construction. Since all fields are checked this additional check acts as a defence in depth.

Recommendation: Explicitly enforce equality between the signed message and the instruction argument, for example:

```
require!(signed_msg == *msg, ErrorCode::MsgMismatch);
```

Adding this will also allow that only the `signed_msg` (or `msg`) needs to have its properties checked with the expected `program_id`, `user_pubkey`, and `expiry_unix` params.

ONR-15

Redundant `expires_at` Parameter in `CreateRedemptionRequest` Due to Solana Replay Protection

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `010057ed9f91711ccf4c6a51c4cbdd1d43a43b18`.

The client provided the following explanation:

Removed the `expires_at` parameter

File(s) affected:

`programs/onreapp/src/instructions/redemption/create_redemption_request.rs`

Description: The `CreateRedemptionRequest` instruction includes an `expires_at` parameter that is validated against the current on chain timestamp to ensure the request has not expired. However, this expiration check is redundant because Solana transactions already include built-in replay protection via recent blockhashes.

In this flow, both the `redeemer` and the `redemption_admin` sign the same transaction containing a recent blockhash. Solana only accepts transactions referencing blockhashes from the last ~150 blocks, which inherently limits how long a signed transaction can be executed.

As a result, the explicit `expires_at` parameter does not provide additional security guarantees for redemption request creation.

Recommendation: Consider removing the `expires_at` parameter and its associated validation logic to simplify the instruction interface and reduce unnecessary complexity.

Auditor Suggestions

S1

Consider Using `UncheckedAccount<'info>` Instead of

Mitigated

`AccountInfo<'info>`

✓ Update

Marked as "Fixed" by the client.

Addressed in: `28da1833e9b02097e1e0b15ebc86375562f767c4` .

Most of the instances of `AccountInfo<'info>` have been replaced with `UncheckedAccount<'info>` . However, some appear to remain.

File(s) affected: `programs/onreapp/src/*`

Description: Throughout the codebase, there are many instances of using `AccountInfo<'info>` for different accounts. However, based on the Anchor framework documentation, new projects should use `UncheckedAccount<'info>` instead.

Recommendation: Consider searching the codebase for any instance of `AccountInfo<'info>` and updating it to `UncheckedAccount<'info>` .

S2 Assign a Default Value to `State.redemption_admin`

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `bdb5b221d6e576a364ac9bd4b6ffd6705652e6b5` .

File(s) affected:

`programs/onreapp/src/instructions/initialization/initialize.rs`

Description: The `initialize()` function initializes the program `State` and sets default values for its fields; however, `redemption_admin` is not initialized.

Recommendation: Consider explicitly setting `redemption_admin` to `Pubkey::default()` within the `initialize()` function.

S3 Properly Validate Ed25519 Program Data

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `8002d3af0091672bdbfb06a5417c16488fea6b20` .

File(s) affected: `programs/onreapp/src/utils/ed25519_parser.rs`

Description: The `parse_ed25519_ix()` function currently ignores the offset and index values provided in the instruction data. However, these values must be validated to ensure they reference the current instruction's data. The Ed25519 program uses `u16::MAX` as a sentinel value to indicate that the signature, public key, and message should be read from the current instruction, preventing unintended data from being loaded from other instructions.

Recommendation: To address this issue, explicitly extract the signature instruction index, public key instruction index, and message instruction index, and validate that each of them is set to `u16::MAX` . This ensures the Ed25519 verification is strictly bound to the current instruction and eliminates the risk of processing unintended or maliciously crafted data.

S4 Code Improvements

Mitigated

✓ Update

Marked as "Fixed" by the client.

Addressed in: `da4c56b281d35676cb3e9d25abfb004e7739e507` .

File(s) affected: `programs/onreapp/src/lib.rs` ,
`programs/onreapp/src/utils/approver/approver_utils.rs` ,
`programs/onreapp/src/instructions/offer/take_offer_permissionless.rs` ,
`programs/onreapp/src/instructions/vault_operations/redemption_withdraw.rs` ,
`programs/onreapp/src/instructions/redemption/make_redemption_offer.rs`

Description: The codebase can benefit from several improvements:

1. **Fixed** In `lib.rs` the comment above `transfer_mint_authority_to_program` states "the PDA is derived from the mint address" is incorrect since only `seeds::MINT_AUTHORITY` is used to derive the `mint_authority` PDA. Consider updating the comment to correctly reflect the PDA derivation seeds.
2. **Fixed** In `take_offer_permissionless.rs` the `take_offer_permissionless` instruction the `state.is_killed` and `state.boss` are validated in the function. However, consider using Anchor constraint validation in the `ctx` struct instead.
3. **Fixed** In `approver_utils.rs` the `verify_approval_message_generic()` function the validation statement `signed_msg.expiry_unix >= Clock::get()?.unix_timestamp` can check against the local variable `now` instead.
4. **Unresolved** In `redemption_withdraw.rs` the `RedemptionVaultWithdraw` struct includes a `boss_token_account` account which is not marked mutable despite receiving token transfers.

5. **Fixed** In `make_redemption_offer.rs` the comment above `token_in_mint` states "This is the token_out_mint from the original offer." which should be updated to state `token_in_mint`.
6. **Fixed** In `make_redemption_offer.rs` the comment above `MakeRedemptionOfferErrorCode::InvalidFee`, should read "maximum allowed value of 1000 (10%)". Moreover the `msg` associated with the error should be updated to `fee_basis_points must be <= 1000`.
7. **Fixed** In `set_redemption_admin.rs` the comment above `set_redemption_admin` references the wrong event name which should be corrected to `RedemptionAdminUpdatedEvent`.
8. **Fixed** In `update_redemption_offer_fee.rs` the comment above `UpdateRedemptionOfferFeeErrorCode::InvalidFee`, should read "maximum allowed value of 1000 (10%)". Moreover the `msg` associated with the error should be updated to `fee_basis_points must be <= 1000`.

Recommendation: Consider applying the code improvements as described.

S5 Missing Validation

Mitigated



Update

Marked as "Fixed" by the client.

Addressed in: `8bb5dfa8912055fef28dcf8fc8fd7d982564a8d6`.

File(s) affected: `programs/onreapp/src/utils/token_utils.rs`,
`programs/onreapp/src/instructions/offer/delete_offer_vector.rs`,
`programs/onreapp/src/instructions/state_operations/set_redemption_admin.rs`

Description: There are several key functions within the codebase that lack proper input parameter validations. A non-exhaustive list of such function parameters is provided below:

1. **Unresolved** In `delete_offer_vector.rs` the validation `vector_start_time > now` should be adjusted to `vector_start_time => now` so that it aligns with the validation in the add vector instruction, `validate_inputs` function.
2. **Fixed** In `set_redemption_admin.rs` the `RedemptionAdminUpdatedEvent` is emitted even during a no-op (e.g. when setting the same admin). Consider validating that the `new_redemption_admin` is different from the current `state.redemption_admin`.
3. **Unresolved** In `set_redemption_admin.rs` the `set_redemption_admin` instruction there is missing validation that the `new_redemption_admin` address is not set to `Pubkey::default()`.
4. **Fixed** In `update_redemption_offer_fee.rs` the `RedemptionOfferFeeUpdatedEvent` is emitted even during a no-op (e.g. when setting the same fee). Consider validating that the `new_fee_basis_points` is different from the current `redemption_offer.fee_basis_points`.
5. **Fixed** In `token_utils.rs` the `execute_token_operations` instruction performs an unchecked addition for calculating amount `params.token_in_net_amount +`

- `params.token_in_fee_amount` which can overflow. Consider using `checked_add` to add these two values.
6. **Fixed** In `token_utils.rs` the `calculate_token_out_amount` instruction is missing a validation for a zero `price`.
 7. **Fixed** In `token_utils.rs` the `calculate_token_out_amount` instruction there is an unchecked cast from `u128` to `u64` that can truncate silently. Consider validating that the value is `<= u64::MAX`.
 8. **Fixed** In `token_utils.rs` the `calculate_token_out_amount` instruction there is an exponentiation using `10_u128.pow(...)` relies on unchecked decimal values, which may overflow or panic if unrealistic decimals are provided. Consider enforcing reasonable bounds on token decimal values.
 9. **Fixed** In `create_redemption_request.rs` the `CreateRedemptionRequest` instruction accepts a mutable `redemption_offer` account but does not explicitly validate that it is the correct or intended redemption offer for the request being created (e.g., specific mints or configuration). Consider adding explicit validation to ensure the provided `redemption_offer` is the intended one.
 10. **Fixed** In `redemption_utils.rs` the redemption calculation computes `token_out_amount` using `u128` arithmetic and then casts the result directly to `u64` without a bounds check. If the computed value exceeds `u64::MAX`, the cast will silently truncate the high bits. Consider before casting, to explicitly check that the computed `u128` value fits within `u64`.

Recommendation: Consider applying the validations as described.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means,

such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

We successfully executed the full test suite using the `anchor test` command at commit `4f65a81be7425ab3ca3d0048a0d7e57475ff2f9e` on a system with the following configuration:

- **OS:** Ubuntu 24.04.3 LTS
- **Rust:** rustc 1.92.0 (ded5c06cf 2025-12-08)
- **Solana:** solana-cli 3.0.13 (src:90098d26; feat:3604001754, client: Agave)
- **AVM:** avm 0.32.1
- **Anchor:** anchor-cli 0.32.1
- **Node.js:** v24.10.0
- **pnpm:** 9.15.4

```
RUN v3.2.4 /root/onre-sol
```

```
✓ tests/market_info/get_nav_adjustment.spec.ts (11 tests) 4378ms
  ✓ Get NAV Adjustment > Basic Functionality Tests > Should return
  positive adjustment for first vector (compared to 0) 1112ms
  ✓ Get NAV Adjustment > Vector Transition Tests > Should calculate
  positive adjustment when price increases between intervals 308ms
  ✓ Get NAV Adjustment > Vector Transition Tests > Should handle
  multiple vector transitions correctly 372ms
  ✓ Get NAV Adjustment > Error Condition Tests > Should fail when no
  vector is active at current time 417ms
  ✓ Get NAV Adjustment > Edge Case Tests > Should handle time
  progression within same vector 451ms
  ✓ Get NAV Adjustment > Edge Case Tests > Should provide consistent
  results for same state 338ms
✓ tests/vault_operations/vault_operations.spec.ts (13 tests) 4578ms
  ✓ Vault Operations > Vault is initialized correctly 789ms
  ✓ Vault Operations > Offer Vault Operations > Deposit tokens to
  offer vault should succeed 408ms
  ✓ Vault Operations > Offer Vault Operations > Non-boss cannot
  deposit to offer vault 300ms
  ✓ Vault Operations > Vault Withdrawal Operations > Offer Vault
  Withdrawals > Withdraw tokens from offer vault should succeed 331ms
  ✓ Vault Operations > Vault Withdrawal Operations > Offer Vault
  Withdrawals > Withdraw Token2022 tokens from offer vault should
  succeed 402ms
  ✓ Vault Operations > Vault Withdrawal Operations > Offer Vault
```

Withdrawals > Withdraw with wrong token_program should fail 343ms

- ✓ Vault Operations > Redemption Vault Operations > Non-boss cannot deposit to redemption vault 304ms
- ✓ Vault Operations > Redemption Vault Withdrawal Operations > Withdraw Token2022 tokens from redemption vault should succeed 301ms
- ✓ tests/offer/delete_offer_vector.spec.ts (10 tests) 4664ms
 - ✓ Delete Offer Vector > Should delete an existing vector from an offer 1088ms
 - ✓ Delete Offer Vector > Should fail with incorrect token mints 353ms
 - ✓ Delete Offer Vector > Should fail when vector start_time is zero 354ms
 - ✓ Delete Offer Vector > Should fail when vector doesn't exist in the offer 368ms
 - ✓ Delete Offer Vector > Should delete specific vector while keeping others 479ms
 - ✓ Delete Offer Vector > Should reject when called by non-boss 343ms
 - ✓ Delete Offer Vector > Previously Active Vector Validation > Should reject deletion of past vector 442ms
 - ✓ Delete Offer Vector > Previously Active Vector Validation > Should reject deletion of current active vector 357ms
 - ✓ Delete Offer Vector > Previously Active Vector Validation > Should allow deletion of future vector 465ms
 - ✓ Delete Offer Vector > Previously Active Vector Validation > Should allow deletion when all vectors are in the future (no active vector) 405ms
- ✓ tests/market_info/get_tvl.spec.ts (12 tests) 5238ms
 - ✓ Get TVL > SPL Token tests > Basic Functionality Tests > Should successfully get TVL for offer with active vector 1070ms
 - ✓ Get TVL > SPL Token tests > Basic Functionality Tests > Should be read-only instruction (no state changes) 398ms
 - ✓ Get TVL > SPL Token tests > TVL Calculation Tests > Should calculate TVL correctly with different prices 357ms
 - ✓ Get TVL > SPL Token tests > TVL Calculation Tests > Should handle 0 APR values correctly 374ms
 - ✓ Get TVL > SPL Token tests > Error Condition Tests > Should fail with non-existent offer 374ms
 - ✓ Get TVL > SPL Token tests > Error Condition Tests > Should fail when offer has no active vectors 317ms
 - ✓ Get TVL > SPL Token tests > Error Condition Tests > Should fail when no vector is active at current time 473ms
 - ✓ Get TVL > SPL Token tests > Error Condition Tests > Should fail with wrong token_out_mint 383ms
 - ✓ Get TVL > SPL Token tests > Edge Case Tests > Should handle multiple vectors and use most recent active one 492ms
 - ✓ Get TVL > SPL Token tests > Edge Case Tests > Should handle very large token supplies 469ms

```
✓ tests/redemption/cancel_redemption_request.spec.ts (11 tests)
7427ms
  ✓ Cancel redemption request > Should cancel redemption request as redeemer 1361ms
  ✓ Cancel redemption request > Should cancel redemption request as redemption_admin 705ms
  ✓ Cancel redemption request > Should cancel redemption request as boss 469ms
  ✓ Cancel redemption request > Should subtract amount from requested_redemptions 470ms
  ✓ Cancel redemption request > Should correctly handle multiple cancellations 800ms
  ✓ Cancel redemption request > Should close the redemption request account and return rent to redemption_admin 311ms
  ✓ Cancel redemption request > Should reject when signer is unauthorized 404ms
  ✓ Cancel redemption request > Should allow cancelling one request while others remain active 782ms
  ✓ Cancel redemption request > Should return locked tokens to redeemer 681ms
  ✓ Cancel redemption request > Should reject when kill switch is activated 735ms
  ✓ Cancel redemption request > Should succeed after kill switch is deactivated 700ms
✓ tests/state_operations/configure_max_supply.spec.ts (14 tests)
7820ms
  ✓ Configure Max Supply and Enforcement > Configure Max Supply Instruction > Boss can configure max supply 958ms
  ✓ Configure Max Supply and Enforcement > Configure Max Supply Instruction > Boss can set max supply to zero (no cap) 372ms
  ✓ Configure Max Supply and Enforcement > Configure Max Supply Instruction > Boss can update max supply multiple times 330ms
  ✓ Configure Max Supply and Enforcement > Mint To Enforcement > Cannot mint ONyc beyond max supply cap 376ms
  ✓ Configure Max Supply and Enforcement > Mint To Enforcement > Can mint exactly up to max supply cap 394ms
  ✓ Configure Max Supply and Enforcement > Mint To Enforcement > Can mint multiple times until reaching cap 551ms
  ✓ Configure Max Supply and Enforcement > Mint To Enforcement > Can mint without limits when max supply is zero 337ms
  ✓ Configure Max Supply and Enforcement > Take Offer Enforcement > Cannot take offer if minting would exceed max supply 554ms
  ✓ Configure Max Supply and Enforcement > Take Offer Enforcement > Can take offer within max supply cap 679ms
  ✓ Configure Max Supply and Enforcement > Take Offer Enforcement > Multiple users can take offers until cap is reached 859ms
  ✓ Configure Max Supply and Enforcement > Take Offer Permissionless Enforcement > Cannot take permissionless offer if minting would exceed
```

max supply 620ms

- ✓ Configure Max Supply and Enforcement > Take Offer Permissionless Enforcement > Can take permissionless offer within max supply cap 624ms
- ✓ Configure Max Supply and Enforcement > Take Offer Permissionless Enforcement > Permissionless offer respects cumulative supply from all minting operations 885ms
- ✓ tests/offer/add_offer_vector.spec.ts (20 tests) 8578ms
 - ✓ Add Offer Vector > Should create an offer and add a time vector 782ms
 - ✓ Add Offer Vector > Should calculate start_time as current time when base_time is in the past 352ms
 - ✓ Add Offer Vector > Should auto-increment vector IDs correctly 399ms
 - ✓ Add Offer Vector > Should reject invalid parameters 601ms
 - ✓ Add Offer Vector > Should allow zero apr 340ms
 - ✓ Add Offer Vector > Should reject start_time before latest existing vector start_time 372ms
 - ✓ Add Offer Vector > Should reject start_time equal to latest existing vector start_time 372ms
 - ✓ Add Offer Vector > Should allow adding vector with base_time before last vector's start_time 377ms
 - ✓ Add Offer Vector > Should reject explicit startTime that equals existing vector's startTime 349ms
 - ✓ Add Offer Vector > Should enforce that explicit startTime cannot create vector ordering violations 512ms
 - ✓ Add Offer Vector > Should reject when offer has maximum vectors 744ms
 - ✓ Add Offer Vector > Should handle large price and apr values correctly 417ms
 - ✓ Add Offer Vector > Should handle minimum valid values (1 for all fields) 359ms
 - ✓ Add Offer Vector > Should reject when called by non-boss 331ms
 - ✓ Add Offer Vector > Should handle vectors added to multiple different offers 387ms
 - ✓ Add Offer Vector > Should clean old past vectors, keeping only current active and previous active 482ms
 - ✓ Add Offer Vector > Should allow to add vector when array is full and cleanup would make space 592ms
- ✓ tests/offer/delete_all_offer_vectors.spec.ts (7 tests) 3798ms
 - ✓ Delete All Offer Vectors > Should delete all vectors from an offer with multiple vectors 997ms
 - ✓ Delete All Offer Vectors > Should succeed when offer has no vectors 332ms
 - ✓ Delete All Offer Vectors > Should delete past, active, and future vectors 518ms
 - ✓ Delete All Offer Vectors > Should delete all MAX_VECTORS vectors 814ms

- ✓ Delete All Offer Vectors > Should fail with incorrect token mints 312ms
- ✓ Delete All Offer Vectors > Should reject when called by non-boss 376ms
- ✓ Delete All Offer Vectors > Should allow adding vectors after deleting all 435ms
- ✓ tests/market_info/get_apy.spec.ts (11 tests) 5014ms
 - ✓ Get APY > Basic Functionality Tests > Should successfully get APY for offer with active vector 961ms
 - ✓ Get APY > Basic Functionality Tests > Should be read-only instruction (no state changes) 396ms
 - ✓ Get APY > APR to APY Conversion Tests > Should handle 0% APR correctly (should return 0% APY) 369ms
 - ✓ Get APY > APR to APY Conversion Tests > Should calculate correct APY for 10% APR 475ms
 - ✓ Get APY > APR to APY Conversion Tests > Should calculate correct APY for higher APR values 474ms
 - ✓ Get APY > APR to APY Conversion Tests > Should handle very small APR values 433ms
 - ✓ Get APY > Error Condition Tests > Should fail with non-existent offer 379ms
 - ✓ Get APY > Error Condition Tests > Should fail when offer has no active vectors 433ms
 - ✓ Get APY > Error Condition Tests > Should fail when no vector is active at current time 365ms
 - ✓ Get APY > Edge Case Tests > Should handle multiple vectors and use most recent active one 374ms
 - ✓ Get APY > Edge Case Tests > Should provide consistent results for same APR 342ms
- ✓ tests/state_operations/remove_approver.spec.ts (12 tests) 5295ms
 - ✓ Remove Approver > Boss can remove first approver successfully 904ms
 - ✓ Remove Approver > Boss can remove second approver successfully 474ms
 - ✓ Remove Approver > Non-boss cannot remove approver – should fail 354ms
 - ✓ Remove Approver > Cannot remove approver that doesn't exist 392ms
 - ✓ Remove Approver > Cannot remove default pubkey 374ms
 - ✓ Remove Approver > Can remove both approvers 391ms
 - ✓ Remove Approver > Can remove and re-add the same approver 428ms
 - ✓ Remove Approver > Boss can remove themselves if they are an approver 446ms
 - ✓ Remove Approver > After removing an approver, can add a new one to the empty slot 349ms
 - ✓ Remove Approver > Removing approver from slot 1 allows adding to that slot 436ms
 - ✓ Remove Approver > Can swap approvers by removing and adding

441ms

- ✓ tests/market_info/get_circulating_supply.spec.ts (5 tests) 2078ms

- ✓ Get Circulating Supply > SPL Token tests > Basic Functionality Tests > Should successfully get circulating supply for offer 679ms

- ✓ Get Circulating Supply > SPL Token tests > Basic Functionality Tests > Should be read-only instruction (no state changes) 350ms

- ✓ Get Circulating Supply > SPL Token tests > Circulating Supply Calculation Tests > Should handle circulating supply calculation with vault logic 542ms

- ✓ Get Circulating Supply > Token2022 tests > Should calculate circulating supply correctly for Token2022 308ms

- ✓ tests/market_info/get_nav.spec.ts (9 tests) 3617ms

- ✓ Get NAV > Basic Functionality Tests > Should successfully get NAV for offer with active vector in first interval 913ms

- ✓ Get NAV > Basic Functionality Tests > Should successfully get NAV for offer with multiple vectors 444ms

- ✓ Get NAV > Basic Functionality Tests > Should be read-only instruction (no state changes) 325ms

- ✓ Get NAV > Price Calculation Tests > Should calculate price after time advancement 348ms

- ✓ Get NAV > Price Calculation Tests > Should handle 0 APR values 419ms

- ✓ Get NAV > Error Condition Tests > Should fail when no vector is active at current time 338ms

- ✓ Get NAV > Edge Case Tests > Should handle multiple vectors and use most recent active one 363ms

- ✓ tests/redemption/create_redemption_request.spec.ts (11 tests) 7140ms

- ✓ Create redemption request > Create redemption request should succeed with valid params 1252ms

- ✓ Create redemption request > Should increment counter after each request 789ms

- ✓ Create redemption request > Should update requested_redemptions in RedemptionOffer 589ms

- ✓ Create redemption request > Should accumulate requested_redemptions from multiple requests 694ms

- ✓ Create redemption request > Should create unique PDAs for different counters 738ms

- ✓ Create redemption request > Redeemer pays for account creation 553ms

- ✓ Create redemption request > Should lock tokens in redemption vault 435ms

- ✓ Create redemption request > Should reject when kill switch is activated 403ms

- ✓ Create redemption request > Should succeed after kill switch is deactivated 556ms

- ✓ Create redemption request > Anyone can create a redemption request without admin approval 604ms

- ✓ Create redemption request > Same redeemer can create multiple requests 505ms
- ✓ tests/offer/take_offer_permissionless.spec.ts (23 tests) 14371ms
 - ✓ Take Offer Permissionless > Basic Functionality Tests > Should successfully execute permissionless offer 1193ms
 - ✓ Take Offer Permissionless > Basic Functionality Tests > Should complete 4-step transfer process correctly 441ms
 - ✓ Take Offer Permissionless > Basic Functionality Tests > Should APR identical results to direct take_offer 645ms
 - ✓ Take Offer Permissionless > Basic Functionality Tests > Should leave intermediary accounts with zero balance after large transaction 673ms
 - ✓ Take Offer Permissionless > Price Calculation Tests > Should calculate correct price in first interval 532ms
 - ✓ Take Offer Permissionless > Price Calculation Tests > Should calculate correct price with fee 597ms
 - ✓ Take Offer Permissionless > Price Calculation Tests > Should maintain price within same interval 635ms
 - ✓ Take Offer Permissionless > Price Calculation Tests > Should calculate higher price in second interval 573ms
 - ✓ Take Offer Permissionless > Price Calculation Tests > Should use most recent active vector 547ms
 - ✓ Take Offer Permissionless > Error Handling Tests > Should fail when offer does not exist 534ms
 - ✓ Take Offer Permissionless > Error Handling Tests > Should fail when no active vector exists 472ms
 - ✓ Take Offer Permissionless > Error Handling Tests > Should fail with insufficient user token balance 604ms
 - ✓ Take Offer Permissionless > Error Handling Tests > Should fail with insufficient vault balance 653ms
 - ✓ Take Offer Permissionless > Edge Cases > Should handle zero APR (fixed price) correctly 531ms
 - ✓ Take Offer Permissionless > Edge Cases > Should handle high APR over long time period with precision 321ms
 - ✓ Take Offer Permissionless > Mint/Transfer Integration Tests > Program lacks mint authority tests > Should transfer token_out tokens from vault to user when program lacks mint authority 573ms
 - ✓ Take Offer Permissionless > Mint/Transfer Integration Tests > Program lacks mint authority tests > Should transfer token_in tokens from user to boss when program lacks mint authority 646ms
 - ✓ Take Offer Permissionless > Mint/Transfer Integration Tests > Program has mint authority tests > Should mint token_out tokens directly to user when program has mint authority 600ms
 - ✓ Take Offer Permissionless > Mint/Transfer Integration Tests > Program has mint authority tests > Should burn token_in tokens when program has mint authority 610ms
 - ✓ Take Offer Permissionless > Mint/Transfer Integration Tests > Edge Cases > Should handle fee calculations correctly when minting

689ms

- ✓ Take Offer Permissionless > Kill Switch Tests > Should reject take_offer_permissionless when kill switch is enabled 845ms

- ✓ Take Offer Permissionless > Kill Switch Tests > Should reject take_offer_permissionless when offer doesn't allow permissionless 666ms

- ✓ Take Offer Permissionless > Kill Switch Tests > Should allow take_offer_permissionless when kill switch is disabled 761ms

- ✓ tests/redemption/make_redemption_offer.spec.ts (10 tests) 4285ms

- ✓ Make redemption offer > Make redemption offer by boss should succeed 753ms

- ✓ Make redemption offer > Make redemption offer by redemption_admin should succeed 426ms

- ✓ Make redemption offer > Make redemption offer should initialize vault token_in account (ONyc) 390ms

- ✓ Make redemption offer > Should reject when called by unauthorized user 406ms

- ✓ Make redemption offer > Should reject duplicate redemption offers 378ms

- ✓ Make redemption offer > Should work with Token2022 as token_in (ONyc as Token2022) 329ms

- ✓ Make redemption offer > Should create multiple redemption offers for different token pairs 632ms

- ✓ Make redemption offer > Should verify redemption offer references correct offer 471ms

- ✓ tests/offer/take_offer.spec.ts (28 tests) 17613ms

- ✓ Take Offer > Price Calculation Tests > Should calculate correct price in first interval 1193ms

- ✓ Take Offer > Price Calculation Tests > Should calculate correct price with fee 720ms

- ✓ Take Offer > Price Calculation Tests > Should use ceiling division for fee – prevents zero fee on small amounts 717ms

- ✓ Take Offer > Price Calculation Tests > Should use ceiling division for fee – large realistic transaction 603ms

- ✓ Take Offer > Price Calculation Tests > Should maintain price within same interval 722ms

- ✓ Take Offer > Price Calculation Tests > Should calculate higher price in second interval 558ms

- ✓ Take Offer > Multiple Vectors Tests > Should use most recent active vector 608ms

- ✓ Take Offer > Error Cases > Should fail when offer does not exist 368ms

- ✓ Take Offer > Error Cases > Should fail when no active vector exists 496ms

- ✓ Take Offer > Error Cases > Should fail with insufficient user token balance 502ms

- ✓ Take Offer > Error Cases > Should fail with insufficient vault balance 609ms

- ✓ Take Offer > Token Transfer Tests > Should correctly transfer tokens between accounts 583ms
- ✓ Take Offer > Token Transfer Tests > Should correctly transfer Token2022 tokens between accounts 654ms
- ✓ Take Offer > Token Transfer Tests > Should correctly handle Token2022 with zero transfer fees 733ms
- ✓ Take Offer > Token Transfer Tests > Should reject token_out with non-zero transfer fees 624ms
- ✓ Take Offer > Token Transfer Tests > Should reject token_in with non-zero transfer fees 842ms
- ✓ Take Offer > Edge Cases > Should not allow take offer when token pair doesn't exist (wrong token_in) 513ms
- ✓ Take Offer > Edge Cases > Should handle zero APR (fixed price) correctly 544ms
- ✓ Take Offer > Edge Cases > Should handle high APR over long time period with precision 649ms
- ✓ Take Offer > Mint/Burn Integration Tests > Program lacks mint authority tests > Should transfer token_out tokens from vault to user when program lacks mint authority 482ms
- ✓ Take Offer > Mint/Burn Integration Tests > Program lacks mint authority tests > Should transfer token_in tokens from user to boss when program lacks mint authority 473ms
- ✓ Take Offer > Mint/Burn Integration Tests > Program has mint authority tests > Should mint token_out tokens directly to user when program has mint authority 628ms
- ✓ Take Offer > Mint/Burn Integration Tests > Program has mint authority tests > Should burn token_in tokens when program has mint authority 607ms
- ✓ Take Offer > Mint/Burn Integration Tests > Edge Cases > Should handle fee calculations correctly when minting 800ms
- ✓ Take Offer > Mint/Burn Integration Tests > Edge Cases > Should handle fee collections when burning 794ms
- ✓ Take Offer > Kill Switch Tests > Should reject take_offer when kill switch is enabled 668ms
- ✓ Take Offer > Kill Switch Tests > Should allow take_offer when kill switch is disabled 559ms
- ✓ tests/redemption/update_redemption_offer_fee.spec.ts (8 tests) 4462ms
 - ✓ Update redemption offer fee > Fee update > Should successfully update redemption offer fee 1194ms
 - ✓ Update redemption offer fee > Fee update > Should update fee to 0 (no fee) 476ms
 - ✓ Update redemption offer fee > Fee update > Should update fee to maximum allowed (10%) 380ms
 - ✓ Update redemption offer fee > Fee update > Should update fee multiple times 544ms
 - ✓ Update redemption offer fee > Error cases > Should fail when fee exceeds 10000 basis points 466ms

- ✓ Update redemption offer fee > Error cases > Should fail when fee is way too high 430ms
- ✓ Update redemption offer fee > Error cases > Should fail when non-boss tries to update 435ms
- ✓ Update redemption offer fee > Fee updates with different values > Should handle fractional percentage fees correctly 528ms
- ✓ tests/state_operations/remove_admin.spec.ts (8 tests) 3860ms
 - ✓ Remove Admin > Boss can remove an admin successfully 917ms
 - ✓ Remove Admin > Non-boss cannot remove admin - should fail 377ms
 - ✓ Remove Admin > Cannot remove admin that doesn't exist 349ms
 - ✓ Remove Admin > Can remove all admins one by one 565ms
 - ✓ Remove Admin > Can remove and re-add the same admin 510ms
 - ✓ Remove Admin > Boss can remove themselves if they are in the admin list 449ms
 - ✓ Remove Admin > Cannot remove the same admin twice 359ms
 - ✓ Remove Admin > Removing admin preserves order of remaining admins 329ms
- ✓ tests/mint_authority/mint_to.spec.ts (7 tests) 2879ms
 - ✓ Mint To > Boss can mint ONyc tokens to their account after transferring mint authority 940ms
 - ✓ Mint To > Non-boss cannot mint ONyc tokens - should fail 411ms
 - ✓ Mint To > Cannot mint without program having mint authority - should fail 315ms
 - ✓ Mint To > Cannot mint with different mint than stored in state - should fail 341ms
- ✓ tests/mint_authority/mint_authority_transfer.spec.ts (7 tests) 2403ms
 - ✓ Mint Authority Transfer > transfer_mint_authority_to_program > Should successfully transfer mint authority from boss to program PDA 747ms
 - ✓ Mint Authority Transfer > transfer_mint_authority_to_boss > Should successfully transfer mint authority from program PDA back to boss 325ms
- ✓ tests/offer/make_offer.spec.ts (10 tests) 3641ms
 - ✓ Make offer > Make an offer should succeed 882ms
 - ✓ Make offer > Make multiple offers should succeed 369ms
 - ✓ Make offer > Make an offer should initialize vault token_in account 385ms
 - ✓ Make offer > Make an offer with invalid token mints should fail 303ms
 - ✓ Make offer > Should reject duplicate offers 340ms
 - ✓ Make offer > Should create offer with permissionless enabled 312ms
- ✓ tests/state_operations/set_onyc_mint.spec.ts (7 tests) 2488ms
 - ✓ Set ONyc Mint > Boss can successfully update ONyc mint 779ms
 - ✓ Set ONyc Mint > Setting ONyc mint preserves other state fields 352ms
 - ✓ Set ONyc Mint > Setting ONyc mint after kill switch operations

```
works 317ms
  ✓ tests/state_operations/propose_and_accept_boss.spec.ts (11 tests) 3685ms
    ✓ Propose and Accept Boss > Boss can propose a new boss successfully 875ms
    ✓ Propose and Accept Boss > Proposed boss can accept and become the new boss 364ms
    ✓ Propose and Accept Boss > Complete two-step transfer workflow 349ms
    ✓ Propose and Accept Boss > First proposed boss cannot accept after proposal is changed 366ms
  ✓ tests/offer/update_offer_fee.spec.ts (8 tests) 2277ms
    ✓ Update Offer Fee > Should successfully update fee for existing offer 596ms
    ✓ Update Offer Fee > Should reject update for non-existent offer 316ms
  ✓ tests/offer/take_offer_with_approval.spec.ts (3 tests) 760ms
  ✓ tests/state_operations/add_approver.spec.ts (9 tests) 1550ms
    ✓ Add Approver > Boss can add first approver successfully 499ms
  ✓ tests/state_operations/kill_switch_disable.spec.ts (6 tests) 1506ms
    ✓ Kill Switch Disable > Boss can disable kill switch successfully 558ms
  ✓ tests/state_operations/add_admin.spec.ts (6 tests) 1429ms
    ✓ Add Admin > Boss can add a new admin successfully 347ms
    ✓ Add Admin > Cannot add more than 20 admins 525ms
  ✓ tests/redemption/fulfill_redemption_request.spec.ts (33 tests) 23434ms
    ✓ Fulfill redemption request > Basic fulfillment > Should successfully fulfill a valid redemption request with burn and mint 1441ms
    ✓ Fulfill redemption request > Basic fulfillment > Should transfer token_in to boss when program lacks mint authority 794ms
    ✓ Fulfill redemption request > Basic fulfillment > Should transfer token_out from vault when program lacks mint authority 837ms
    ✓ Fulfill redemption request > Basic fulfillment > Should close redemption request account and return rent to redemption_admin 820ms
    ✓ Fulfill redemption request > Redemption statistics > Should update executed_redemptions in RedemptionOffer 774ms
    ✓ Fulfill redemption request > Redemption statistics > Should decrement requested_redemptions in RedemptionOffer 602ms
    ✓ Fulfill redemption request > Redemption statistics > Should accumulate executed_redemptions from multiple fulfillments 1017ms
    ✓ Fulfill redemption request > Access control > Should reject when redemption_admin is not authorized 786ms
    ✓ Fulfill redemption request > Access control > Should reject when kill switch is activated 887ms
    ✓ Fulfill redemption request > Access control > Should reject when request has already been fulfilled 869ms
```

- ✓ Fulfill redemption request > Price calculation > Should use current price from offer for redemption 579ms
- ✓ Fulfill redemption request > Price calculation > Should handle fractional price 1.003 correctly 691ms
- ✓ Fulfill redemption request > Price calculation > Should handle tokens with different decimals (9 vs 6) 819ms
- ✓ Fulfill redemption request > Price calculation > Should handle high precision price 3.141592653 788ms
- ✓ Fulfill redemption request > Price calculation > Should handle very small amounts correctly 982ms
- ✓ Fulfill redemption request > Price calculation > Should handle price with many decimal places 0.123456789 766ms
- ✓ Fulfill redemption request > Price calculation > Should handle APR-based price growth correctly 901ms
- ✓ Fulfill redemption request > Fee handling > Should correctly deduct fee and transfer to boss when program has mint authority 1018ms
- ✓ Fulfill redemption request > Fee handling > Should transfer full amount (net + fee) to boss when program lacks mint authority 1009ms
- ✓ Fulfill redemption request > Fee handling > Should handle zero fee correctly 733ms
- ✓ Fulfill redemption request > Fee handling > Should correctly calculate fees with non-zero APR and time passage 679ms
- ✓ Fulfill redemption request > Fee handling > Should handle high fee percentage with APR correctly 736ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #1: APR=5%, Fee=1%, 30 days 645ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #2: APR=15%, Fee=5%, 90 days 639ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #3: APR=25%, Fee=2.5%, 180 days 487ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #4: APR=10%, Fee=0.5%, 45 days 513ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #5: APR=30%, Fee=7%, 60 days 499ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #6: APR=8%, Fee=3%, 15 days 406ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #7: APR=12%, Fee=4.5%, 120 days 374ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #8: APR=18%, Fee=6%, 270 days 373ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #9: APR=22%, Fee=1.5%, 365 days 325ms
- ✓ Fulfill redemption request > Token2022 with random APR and fees > Token2022 Test #10: APR=35%, Fee=8%, 7 days 332ms
- ✓ tests/state_operations/close_state.spec.ts (4 tests) 685ms
- ✓ tests/state_operations/kill_switch_enable.spec.ts (6 tests) 946ms
- ✓ Kill Switch Enable > Boss can enable kill switch successfully

```
303ms
✓ tests/state_operations/initialize_kill_switch.spec.ts (4 tests)
473ms
✓ tests/state_operations/set_redemption_admin.spec.ts (2 tests) 324ms

Test Files 33 passed (33)
Tests 346 passed (346)
Start at 00:52:55
Duration 28.65s (transform 2.20s, setup 0ms, collect 47.35s, tests
162.70s, environment 26ms, prepare 9.63s)
```

Changelog

- 2025-12-20 - Initial report
- 2026-01-23 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1100 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland

- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republicc, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor

guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

